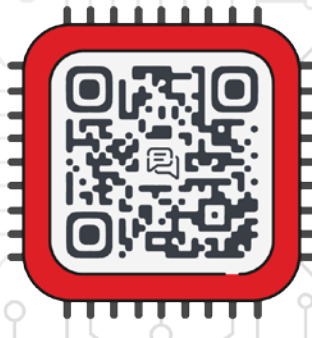




6

11

1650 sp

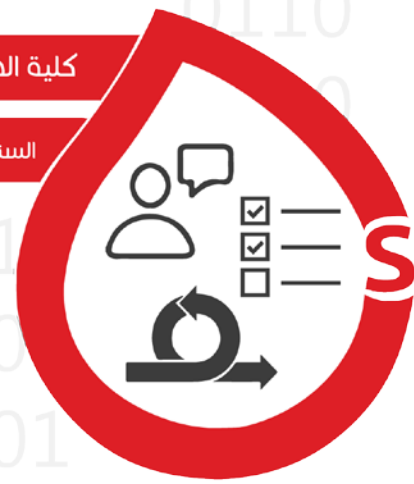
عملي

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Software Testing & Test Cases

عملي مشترك



محتوى مجاني غير مخصص للبيع التجاري
RB Informatics; 02/06/2026

هندسة برمجيات ٢

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

في المحاضرات السابقة تحدثنا عن مفهوم FP, UCP بوصفه أحد أهم الأساليب المستخدمة في هندسة البرمجيات لتقدير حجم الأنظمة البرمجية وقياس وظائفها من منظور المستخدم النهائي، حيث تساعد نقاط الوظائف فريق التطوير على تقدير الجهد والوقت، والتكلفة اللازمة لتنفيذ المشروع البرمجي بصورة أكثر دقة. أما اليوم سنتقل لمرحلة جديدة مهمة وهي اختبار البرمجيات Software Testing.

اختبار البرمجيات Software Testing

يعد اختبار البرمجيات أحد أهم العمليات الأساسية في حياة البرنامج (Software Development Life Cycle - SDLC). حيث يهدف إلى التحقق من أن النظام:

- يعمل بالشكل المطلوب.
- يعمل دائماً.
- يعمل في البيئة التي صمم لها.
- يلبي احتياجات المستخدمين.
- يحقق المتطلبات التي تم تحديدها أثناء مراحل التحليل والتصميم.

في المشاريع البرمجية الحديثة لا يقتصر دور المختبر (Tester) على تجربة النظام والبحث عن الأخطاء فقط، بل أصبح جزءاً أساسياً من عملية تطوير البرمجيات، يشارك في تحليل المتطلبات ودراسة السيناريوهات المختلفة وتصميم حالات الاختبار والتأكد من جودة المنتج قبل وصوله إلى المستخدم النهائي.

ولكن لماذا؟



تكمُن أهمية اختبار البرمجيات في أنه يساعد على:

1. اكتشاف الأخطاء البرمجية (Bugs) مبكراً.

2. تقليل تكلفة الإصلاح.

كلما أُكتشف الخطأ في مراحل مبكرة، انخفضت تكلفة معالجته، لا سيما وأن بعض الأخطاء البرمجية قد تكلف خسائر مالية إذا وصلت إلى بيئة التشغيل الفعلي.

3. تحسين جودة المنتج.

بعض الأخطاء قد تعرض حياة المستخدمين للخطر (مثلاً في الأنظمة الطبية).

4. زيادة رضا المستخدم.

5. التأكد من مطابقة النظام لمتطلبات العميل.



وقد تم إثبات أن تكلفة اكتشاف الخطأ بعد إطلاق النظام قد تكون أعلى بعشرات المرات من تكلفة اكتشافه أثناء التطوير. لذلك أصبحت عملية الاختبار أحد الأعمدة الرئيسية في هندسة البرمجيات الحديثة.

أولاً: مفاهيم اختبار البرمجيات:

هو عملية منهجية تهدف إلى تقييم النظام البرمجي من خلال تنفيذ مجموعة من الاختبارات للتحقق من صحة وظائفه وسلوكه.

1. Tester:

يظن البعض أن الـ tester شخص يجرب البرنامج فقط، لكن دور المختبر يتمثل في:

1. تحليل المتطلبات (Requirements Analysis).

2. دراسة حالات الاستخدام (Use Cases).

3. تصميم حالات الاختبار (Test Cases).

4. تحديد السيناريوهات الإيجابية والسلبية.

5. اكتشاف الأخطاء والمخاطر المحتملة.

6. التحقق من توافق النظام مع احتياجات المستخدم.

ملاحظة:

لا نقوم بالاختبار فقط عند الانتهاء من النظام، بل في كل دورة من دورة حياة النظام: أثناء التحليل و التصميم والتنفيذ وبعد التسليم.

2. الـ Tester هو شخص مختلف عن الـ Developer:

Developer

- شخص يبني النظام.
- يهتم بالتنفيذ.
- يفكر بكيفية عمل النظام.
- يصمم الاختبار.

Tester

- شخص يختبر النظام.
- يهتم بالجودة.
- يفكر بكيفية فشل النظام.
- يكتب الـ code.



3. الـ Testing ≠ الـ Debugging:

عملية اكتشاف الـ bugs (testers).

Testing

عملية تصليح تلك الـ bugs (developers).

Debugging



إذاً مسار العمل بين الـ tester & developer تكون:



4. السيناريو السعيد Happy Path:

هو السيناريو التي تسير فيه جميع الخطوات بصورة صحيحة، مثلاً:

- الحساب صالح.
- المنتج متوفر.
- قاعدة البيانات تعمل.
- الاتصال بالشبكة مستقر.

5. السيناريو السلبي Negative Scenarios:

- كلمة المرور خاطئة.
- حساب محظور.
- انقطاع الإنترنت.
- امتلاء قاعدة البيانات.
- تجاوز الحد المسموح.

ال Tester لا يختبر ال Happy Scenario، بل يبحث عن الحالات الغير متوقعة أو الخاطئة.

6. مراجعة بعض المصطلحات:

1. المتطلبات Requirements:

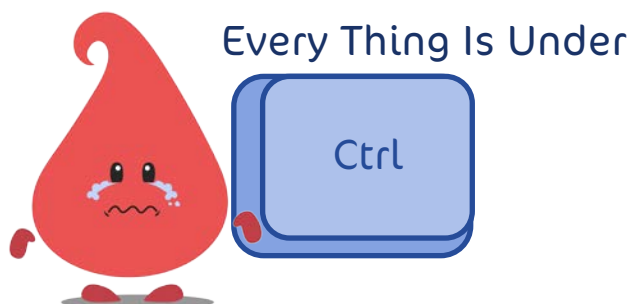
هي المواصفات التي يطلبها العميل أو المستخدم من النظام وتنقسم إلى:

متطلبات وظيفية: هي التي تحدد الوظائف التي يجب أن ينفذها النظام.

متطلبات غير وظيفية: هي التي تحدد خصائص الجودة.

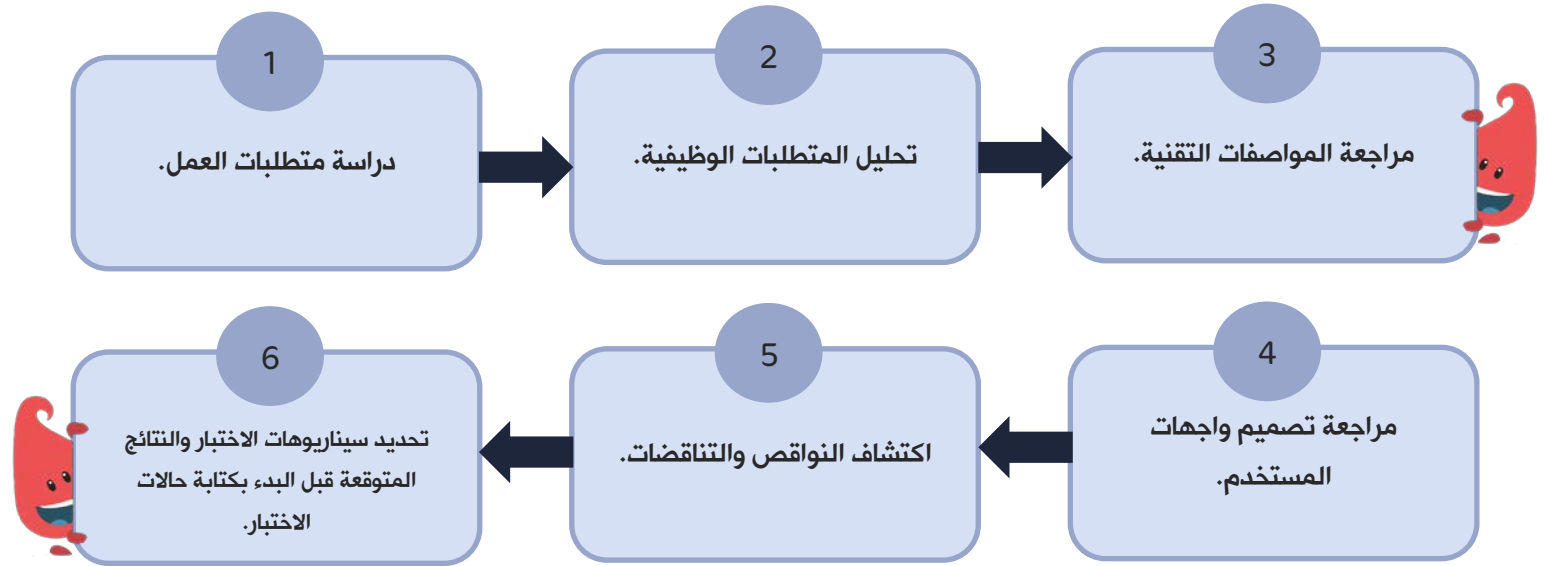
2. حالات الاستخدام Use Cases:

هي وصف للتفاعل بين المستخدم والنظام بهدف تنفيذ وظيفة محددة.



7. فهم المتطلبات:

قبل كتابة أي اختبار يجب فهم المتطلبات بدقة، تمر عملية فهم المتطلبات بالمراحل التالية:



8. العلاقة بين Requirements, Use Cases, Test Cases:

المتطلب يحدد المطلوب، وحالة الاستخدام توضح كيفية التنفيذ، أما حالة الاختبار بناءً على المتطلبات وحالات الاستخدام فتتحقق من أن التنفيذ يعمل بشكل صحيح.

9. مصدر الحقيقة هو الزبون:

في أي مشروع برمجي، يكون العميل هو المصدر الحقيقي للمتطلبات، لذلك أثناء الاختبار لا يقاس نجاح المتطلب فقط بخلوه من الأخطاء، بل بمدى تحقيقه للأهداف التي يريدها العميل.

10. قابلية إعادة إنتاج الخطأ (Reproducibility):

عند اكتشاف خطأ يجب توثيقه بدقة:

- ليس: التطبيق يتعطل أحياناً.
- بل: التطبيق يتعطل عند تنفيذ عملية Check-in عند الساعة 12:01 am.
- كي يستطيع المطور إعادة تنفيذ المشكلة.



لا توجد أخطاء برمجية عارضة تحدث لمرة واحدة دون سبب؛ بل يجب تحديد خطوات تكرار الخطأ لإعادة إنتاجه.
وتحديداً: كلما تم إدخال هذه المدخلات، تظهر دائماً هذه النتيجة الخاطئة.

ثانياً: حالات الاختبار Test Cases:

طوال الوقت كنا نتحدث عن "يكتب اختبار" و "حالة اختبار" أو "سيناريو اختبار".

فما هي حالات الاختبار؟

تحتوي حالة اختبار على:

- سلسلة من الخطوات التي نريد القيام بها (في هذه الحالة).
- بيانات لنستخدمها (دخل).
- خرج متوقع (لكل خطوة نقوم بها).

تعتمد كتابة حالات الاختبار على 3 مصادر أساسية:



تتكون عادةً من:

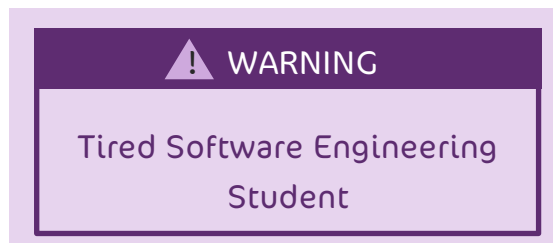
- معرف حالة الاختبار (Test Case ID).
- عنوان الاختبار.
- الوصف (description).
- الشروط المسبقة (preconditions).
- بيانات الاختبار (test data).
- خطوات التنفيذ (steps).
- النتائج المتوقعة (expected results).

- النتائج الفعلية (actual results).
- حالة الاختبار (status).

مثال على حالة اختبار:

Id فريد : عنوان حالة الاستخدام	
الهدف	جملة قصيرة أو جملتين (إذا أصبحت حالة الاختبار طويلة، فإنه يتم تفكيكها وتجزئتها إلى حالات اختبار أخرى).
متطلب سابق	الافتراضات التي يجب تحققها قبل حالة الاختبار.
بيانات الاختبار	■ قائمة المتغيرات وقيمها المحتملة، التي سنستخدمها في حالة الاختبار. ■ قد تكون قيمة محددة أو مجال قيم. ■ يجب إجراء حالة الاختبار مرة لكل مجموعة قيم.
خطوات التنفيذ	خطوات إجراء الاختبار.

TC-001: تسجيل الدخول باستخدام بيانات صحيحة	
الهدف	التحقق من إمكانية تسجيل الدخول باستخدام بيانات صحيحة.
الشروط المسبقة	وجود حساب فعال داخل النظام.
بيانات الاختبار	loginID = {Valid loginID, invalid loginID, valid email, invalid email, empty} password = {valid, invalid, empty}
خطوات التنفيذ	فتح صفحة تسجيل الدخول. إدخال اسم المستخدم. إدخال كلمة المرور. الضغط على زر تسجيل الدخول.
النتيجة المتوقعة	يتم الانتقال إلى الصفحة الرئيسية بنجاح.



ثالثاً: Test Suite:

هي مجموعة من حالات الاختبار المرتبطة بميزة واحدة مثلاً :

Login Test Suite

تحتوي على:

- TC1: تسجيل دخول صحيح.
- TC2: مستخدم غير موجود.
- TC3: كلمة مرور خاطئة.
- TC4: كلمة مرور خاطئة مرتين.
- TC5: كلمة مرور خاطئة 3 مرات.

رابعاً: Requirement Traceability Matrix (RTM):

جدول يربط المتطلبات بحالات الاختبار، مثال:

Requirement	Test Cases
R1	TC1
R2	TC2,TC3
R3	TC4
R4	TC5

فوائده:

1. ضمان تغطية جميع المتطلبات وعدم إهمال أي متطلب.
2. اكتشاف المتطلبات غير المختبر.

خامساً: مشكلات الوقت والتاريخ:

تعد الأخطاء المرتبطة بالوقت والتاريخ، من أكثر الأخطاء انتشاراً في الأنظمة البرمجية، مثلاً:

- التوقيت الصيفي.
- التوقيت الشتوي.
- اختلاف المناطق الزمنية.
- نهاية الشهر.
- نهاية السنة.

قد تؤدي إلى ظهور أخطاء لا تظهر أثناء الاختبارات العادية.

■ مثال على الخطأ الزمني:

موعد الحافلة 8:00

نريد إرسال إشعار الساعة : 7:55

عند تغير التوقيت الشتوي ببعض الدول سيتم إرسال الإشعار الساعة 6:55 وهو وقت خاطئ .

معلومة إضافية من الأستاذ أنور:

التوقيت العالمي UTC:

الـ UTC هو نظام يعتمد على خطوط الطول في الكرة الأرضية، حيث تبلغ قيمة خط غرينيتش قيمته 0، ويحدث انزياح بمقدار +5 أو -5 لكل خط طول.

مثلاً سوريا : +3 UTC بعدها عن خط غرينيتش هو 3

إذاً الـ UTC Offset يمثل الفرق الزمني بين منطقة معينة وبين التوقيت العالمي.

أفضل الممارسات في الأنظمة العالمية هي تخزين جميع التواريخ والأوقات بصيغة UTC داخل قاعدة البيانات، ثم تحويلها إلى التوقيت المحلي المستخدم عند العرض.
فوائد ذلك :

■ توحيد البيانات.

■ تقليل أخطاء المناطق الزمنية.

■ دعم المستخدمين حول العالم.

حينما أقول أن الساعة هي الثامنة صباحاً، فإنني أقصد الساعة الثامنة بتوقيت سوريا، والتي تعادل الساعة الثانية ظهراً في اليابان، على الرغم من حدوثهما في اللحظة الزمنية ذاتها .



سادساً: إعادة إصلاح الخطأ:

عند إصلاح خطأ معين قد يؤثر التعديل على أجزاء أخرى من النظام، لذلك يتم تنفيذ اختبار الـ regression testing للتأكد أن الوظائف السابقة مازالت تعمل بصورة صحيحة.
بدلاً من إعادة اختبار النظام بالكامل، يتم اختبار المناطق الأكثر تأثراً بالتعديل، مما يوفر الوقت والجهد.
(ستحدث عنه في المحاضرة القادمة).

سابعاً: لماذا يعد المختبر المحترف نادراً؟

لأنه يحتاج إلى:

- فهم العمل.
- فهم المتطلبات.
- فهم البرمجة.
- التفكير التحليلي.
- توقع الأخطاء المستقبلية.
- فهم سلوك المستخدم.

وهذه المهارات لا تكتسب إلا من خلال خبرة عملية طويلة ومهارات تحليلية عالية.

خلاصة:

اختبار البرمجيات ليس مجرد عملية لاكتشاف الأخطاء، بل هو نشاط هندسي متكامل يهدف إلى ضمان جودة المنتج البرمجي وتحقيق متطلبات العميل.

ويعتمد نجاح عملية الاختبار على الفهم العميق للمتطلبات وحالات الاستخدام، والقدرة على تصميم حالات اختبار فعالة تشمل السيناريوهات الإيجابية والسلبية، إضافة إلى مراعاة الجوانب الأمنية والزمنية والتقنية الغير متوقعة .

كما تشكل حالات الاختبار ومجموعات الاختبار ومصفوفات التتبع أدوات أساسية لضمان شمولية الاختبار ورفع مستوى جودة الأنظمة البرمجية.

